Week 4, Day 6 (Weekend): Project Polish, Resilience, Observability, README and GitHub

Prepared for Upendra Kumar Sharma | 5 to 6 hours | Goal: finish the project so that a stranger can run it in minutes, trust it, and you can defend every decision in an interview

Today's Schedule

- **0:00 to 1:15** Add Resilience4j to the one synchronous call and test it (section 1).
- **1:15 to 2:15** Add Actuator, Micrometer metrics, tracing and a Prometheus and Grafana dashboard (section 2).
- **2:15 to 3:00** Harden the services and add CI (sections 3 and 4).
- **3:00 to 4:15** Write the README, the architecture diagram and the decision records (section 5).
- **4:15 to 4:45** Publish the repository properly and update your resume (sections 6 and 7).
- **4:45 to 5:30** Practice the project pitch and the deep dive (section 8).
- **5:30 to 5:50** Review the checklist (section 10).

A finished, documented, tested project of modest size is worth more than a large unfinished one. Cut scope before you cut quality.

1. Resilience4j on the Synchronous Call

Which call? Most of the system is asynchronous. The one synchronous dependency is order-service asking inventory-service for a **stock availability hint** before accepting an order (the Feign client from Week 3, Day 2).

The design decision (put it in the README): this check is **advisory**. The real decision is made later by the saga, with the atomic stock update. So if inventory is slow or down, order-service **still accepts the order** and marks availability as unknown. A resilience feature must always be tied to a decision about what the user sees.

```
@Service
public class AvailabilityService {
    private final InventoryClient inventoryClient;
    public AvailabilityService(InventoryClient inventoryClient) {
this.inventoryClient = inventoryClient; }

    @CircuitBreaker(name = "inventoryCheck", fallbackMethod = "unknown")
    @Bulkhead(name = "inventoryCheck")
    public Availability check(String sku) {
        return inventoryClient.getStock(sku).toAvailability();
    }

    private Availability unknown(String sku, Throwable ex) {
        return Availability.UNKNOWN;       // the saga makes the final
decision
    }
}
```

```
resilience4j:
  circuitbreaker:
    instances:
      inventoryCheck:
        slidingWindowSize: 20
        minimumNumberOfCalls: 10
        failureRateThreshold: 50
        slowCallDurationThreshold: 1s
        slowCallRateThreshold: 80
        waitDurationInOpenState: 20s
        permittedNumberOfCallsInHalfOpenState: 3
        ignoreExceptions:
          - com.example.order.StockNotFoundException
  bulkhead:
    instances:
      inventoryCheck:
        maxConcurrentCalls: 20
        maxWaitDuration: 0
```

- Keep the Feign **connect and read timeouts** short (for example 500 ms and 1 s), so the breaker sees slow calls quickly.
- Add the dependencies from Week 3, Day 3 (resilience4j-spring-boot3, spring-boot-starter-aop, actuator).
- Also review the other failure points. Kafka sends use bounded timeouts, Redis calls have short timeouts and fail open, and consumers use retries and a dead letter topic.

Test it with a fake inventory service (WireMock)

- Stub the inventory endpoint to return 500 (or to respond slowly).
- Call the check 12 times, then assert that the breaker state is **OPEN** (read it from the CircuitBreakerRegistry), and that further calls return UNKNOWN **without reaching WireMock** (verify the request count stops growing).
- Remove the failure, wait for the open duration, and assert that the breaker returns to CLOSED.
- Use Awaitility for the waiting, not Thread.sleep.

2. Observability: Health, Metrics, Traces and Logs

Actuator and Micrometer

```
management:
  server:
    port: 9091                      # internal-only management port
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus,circuitbreakers
  endpoint:
    health:
      probes:
        enabled: true               # liveness and readiness endpoints
  metrics:
    tags:
      application: ${spring.application.name}
```

- **Liveness vs readiness:** liveness says the process is alive (restart it if not). Readiness says it can take traffic (the database and Kafka are reachable). Container platforms use them separately. Use the readiness endpoint in your Compose healthchecks.
- **Keep the management port off the public network.** Never expose Actuator through the gateway. Only Prometheus on the internal network scrapes it.
- Add micrometer-registry-prometheus so that /actuator/prometheus serves the metrics.
- **Metrics you get for free:** JVM memory and garbage collection, HTTP server request timers (with percentiles if enabled), HikariCP pool usage, Resilience4j breaker state, and Kafka client metrics (check /actuator/metrics for the names).

Custom business metrics (these are what make a dashboard meaningful)

```
@Component
public class OrderMetrics {
    private final Counter created;
    private final Counter rejected;

    public OrderMetrics(MeterRegistry registry, OutboxRepository outbox) {
        this.created = Counter.builder("orders.created").register(registry);
        this.rejected = Counter.builder("orders.rejected").tag("reason",
"insufficient_stock").register(registry);
        Gauge.builder("outbox.unpublished", outbox,
OutboxRepository::countUnpublished)
             .description("Outbox rows waiting to be published")
             .register(registry);
    }
    public void orderCreated()  { created.increment(); }
    public void orderRejected() { rejected.increment(); }
}
```

- The **outbox backlog gauge** is the single most useful reliability metric in this design. A growing number means the relay or Kafka is in trouble.
- Keep metric **tag values low in cardinality** (statuses and reasons, never user ids or order ids), or you will overload the metrics system.

Distributed tracing

- Add a Micrometer Tracing bridge (Brave or OpenTelemetry) and an exporter (Zipkin or OTLP), and run **Zipkin** or Jaeger in Compose. Set the sampling probability to 1.0 for development and lower it for real traffic.
- Enable observation for Kafka (the Spring Kafka template and listener observation settings) so the **trace context travels in message headers** and one trace follows a request from the gateway through order-service, Kafka, inventory-service and back.
- Put the **trace id in every log line**. The property name for the log pattern depends on your Spring Boot version, so check the documentation.

Prometheus and Grafana in Compose

```
  prometheus:
    image: prom/prometheus
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
    ports:
      - "9090:9090"
  grafana:
    image: grafana/grafana
    ports:
      - "3000:3000"
  zipkin:
    image: openzipkin/zipkin
    ports:
      - "9411:9411"
```

Pin each image to a current version tag instead of leaving it unversioned.

```
# prometheus/prometheus.yml
global:
  scrape_interval: 15s
scrape_configs:
  - job_name: services
    metrics_path: /actuator/prometheus
    static_configs:
      - targets:
          - api-gateway:9091
          - auth-service:9091
          - order-service:9091
          - inventory-service:9091
          - notification-service:9091
```

Dashboard panels to build (one screenshot goes in the README)

- Request rate, error rate and p95 latency per service.
- Orders created and rejected over time.
- **Outbox backlog** and the dead letter topic count.
- Consumer lag (from a Kafka exporter, or from the group describe command).
- Circuit breaker state, cache hit ratio and rate limiter rejections.

Alerts to describe in the README (even if you do not deploy them): outbox backlog above a threshold for 5 minutes, any new dead letter message, consumer lag growing, a breaker open for more than 2 minutes, a 5xx rate above 1 percent, and p95 latency above your target.

3. Hardening Checklist

- **Graceful shutdown:** server.shutdown=graceful with a shutdown timeout, so in-flight requests finish and Kafka consumers commit before the container stops.
- **Healthchecks in Compose** using the readiness endpoint, and depends_on with the service_healthy condition.
- **Resource limits** and a JVM memory setting that respects them (MaxRAMPercentage).
- **Secrets:** only from environment variables or secret files, never in images or Git. Rotate the JWT secret value you used in development.
- **Input limits:** maximum request size, a maximum page size, and bounded collections in request bodies.
- **Security:** only the gateway is published, services also validate tokens, CORS lists explicit origins, and security headers are on.
- **Timeouts everywhere:** HTTP clients, database connections, Redis and Kafka.
- **Images:** multi-stage builds, a non-root user, a .dockerignore file, and pinned base image versions. Scan images and dependencies for known vulnerabilities (for example with Trivy or the OWASP dependency check).
- **Database:** Flyway migrations only (Hibernate set to validate), plus the indexes you designed.

4. Continuous Integration with GitHub Actions

A green build badge tells a reviewer that the tests run on a clean machine. GitHub's hosted Ubuntu runners include Docker, so **Testcontainers tests run there**.

```
name: CI
on:
  push:
    branches: [main]
  pull_request:

jobs:
  build:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        service: [api-gateway, auth-service, order-service, inventory-
service, notification-service]
    defaults:
      run:
        working-directory: services/${{ matrix.service }}
    steps:
```

```
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: 21
          cache: maven
      - run: mvn -B verify
```

- Check the current major versions of the actions you use. Add a second job that builds the Docker images (without pushing), or pushes them to GitHub Container Registry on tagged releases.
- Keep the build fast. If it takes more than about 10 minutes, find the slow tests.

5. The README, the Diagram and Decision Records

Architecture diagram. GitHub renders **Mermaid** diagrams written in fenced code blocks, so the diagram stays in the repository as text.

```
flowchart LR
  Client --> GW[API Gateway]
  GW --> AUTH[Auth Service]
  GW --> ORD[Order Service]
  GW --> INV[Inventory Service]
  ORD --> ODB[(Orders DB)]
  INV --> IDB[(Inventory DB)]
  ORD -- outbox --> K{{Kafka}}
  K -- orders.created --> INV
  INV -- outbox --> K
  K -- inventory events --> ORD
  K -- order outcome --> NOT[Notification Service]
  INV --> R[(Redis)]
  GW --> R
```

In the README, put this inside a code block with the language set to mermaid. Add a second diagram, a **sequence diagram of the order saga**, covering the happy path and the rejection path.

README structure (a reviewer reads the top first)

1. **Title, one-line description and the CI badge.**
2. **What it demonstrates:** outbox, idempotent consumers, saga, JWT at the edge, caching, rate limiting, resilience, observability.
3. **Architecture:** the diagrams, then one paragraph per service.

4. **Quick start:** copy .env.example to .env, then one docker compose command, then the URLs (gateway, Grafana, Zipkin).
5. **Try it:** the curl commands for login, creating an order, reading it, and the rejection case.
6. **Design decisions and trade-offs:** the part that impresses. Database per service, no Eureka or config server and why, HS256 now and RS256 later, no shared event library, advisory stock check with a fallback, at-least-once delivery with idempotent handlers, cache never used for reservation decisions.
7. **Reliability:** the failure drill table (what you broke, what happened, how it recovered).
8. **Observability:** a dashboard screenshot and the metric names.
9. **Testing:** how to run the tests, and what is covered.
10. **Performance notes:** your **own measured numbers**, labeled as local measurements (for example cache hit versus miss latency, and throughput from the producer test).
11. **Limitations and next steps:** Schema Registry, RS256 with JWKS, Kubernetes manifests, saga timeouts and a stuck order sweeper, a payment service with a real compensation, load testing at scale.

Architecture Decision Records (ADRs). Add a docs/adr folder with short files, one per important decision. Each has three parts: **Context** (the problem), **Decision** (what you chose) and **Consequences** (what you gained and gave up). Example titles: 0001 Use the transactional outbox, 0002 Database per service, 0003 JWT validation at the gateway and in services, 0004 Cache is advisory only. ADRs show senior-level thinking, and they double as your interview notes.

6. Publishing the Repository Properly

- Check the repository clean: no secrets, no build output, a proper .gitignore, a LICENSE file and a .env.example.
- **Clone it fresh into a new folder and follow your own README exactly.** Fix every place where it fails. This is the single most valuable check.
- Write a clear description and add topics (java, spring-boot, microservices, kafka, redis, docker).
- Tag a release (v1.0.0) and **pin the repository** on your GitHub profile.
- Keep the commit history tidy, with meaningful messages.
- Put a short demo in the README: a screenshot or a short recording of an order moving from PENDING to CONFIRMED, and the Grafana dashboard.

7. Update Your Resume and Profile

A draft Projects section, to be filled with **your own measured numbers** (do not claim anything you did not build or measure):

- **Order Platform: event-driven microservices** | Java 21, Spring Boot 3, Spring Cloud Gateway, Kafka, PostgreSQL, Redis, Docker | GitHub link
  - Built a five-service order platform behind an API gateway with JWT validation at the edge and in each service, and a database per service.
  - Implemented the transactional outbox, idempotent consumers and an order saga, and verified no lost or duplicate effects across [N] failure drills (broker outage, crash after commit, duplicate delivery).
  - Added Redis cache-aside for stock reads (p95 latency from [X] ms to [Y] ms in local tests), token-bucket rate limiting, and a Resilience4j circuit breaker and bulkhead.
  - Wrote [N] unit and Testcontainers integration tests, CI with GitHub Actions, and Prometheus and Grafana dashboards with tracing.

Also update the skills section **only** with technologies you used in this project (Redis, Testcontainers, Prometheus, Grafana, GitHub Actions), and add your GitHub link near your contact line. Add the repository to the Featured section on LinkedIn. Say "update my resume with the project" and I will rewrite it with these changes.

8. The Project Pitch and Deep Dive

The 3-minute pitch

1. **What and why (20 seconds):** an order platform I built to practice real distributed systems problems.
2. **Architecture (40 seconds):** draw the diagram. Name the services and how they communicate.
3. **The three hardest problems (90 seconds):** reliable event publishing (outbox), duplicate messages (idempotent consumers), and protecting against a slow dependency (circuit breaker and fallback). Give one concrete example for each.
4. **Proof (20 seconds):** the tests, the failure drills and the dashboard.
5. **What you would do next (10 seconds).**

Be ready to go deeper on any box in the diagram for 10 minutes. Practice drawing the saga sequence on paper from memory, and answering what happens at each step if the service crashes.

9. Questions to Practice Aloud

1. Walk me through what happens from the moment a user places an order until it is confirmed.
2. What would happen if Kafka went down for 10 minutes? Show me where the system protects itself.
3. How do you know your consumers handle duplicates correctly?
4. Why did you choose the outbox pattern over publishing directly after commit?
5. Why is the stock check advisory? What would change if it were authoritative?
6. How would you deploy this on Kubernetes, and what would change?
7. What is the weakest part of your design, and how would you fix it?
8. How would this scale to 100 times the traffic?
9. How would you add a payment service and its compensation?
10. What did you learn that surprised you?

10. End-of-Day Checklist

- [ ] The circuit breaker, bulkhead and fallback are in place and tested with a failing fake service.
- [ ] Health probes, Prometheus metrics, a Grafana dashboard and traces work in Compose.
- [ ] CI runs the tests on GitHub and shows a green badge.
- [ ] The README has diagrams, a quick start, design decisions, drill results and your measured numbers.
- [ ] At least three ADRs are written.
- [ ] A fresh clone runs successfully by following the README alone.
- [ ] The repository is tagged, pinned and described, and the resume has the Projects entry.
- [ ] I can give the 3-minute pitch and handle the ten questions without notes.

Tomorrow (Week 4, Day 7, weekend): system design practice (a rate limiter), a mock interview and your Week 4 review.